

Lab 3: Architecture Justification

Kabir Raju G | PES1UG24CS210 | Courier Dispatch System

Architecture Selection: We chose Microservices Architecture for the Courier Dispatch System.

Architectural Choice

Microservices. The system is split into independently deployable services (Dispatch, Tracking, Verification and Notification) behind one API Gateway. The Sender App and Rider App talk only to the gateway, and the services talk to each other through the interfaces shown in the component diagram (IDispatch, ITracking, IVerify, IClientAPI, INotify).

Other styles considered: Layered is easy to follow, but the layers are tightly coupled and one busy part cannot be scaled alone. Client-Server is simple to deploy, but the central server is a single point of failure and a bottleneck once many riders send location updates.

Two Reasons

1. **Independent scaling.** Riders send location updates every few seconds while senders watch the parcel, so Tracking gets far more traffic than Dispatch or Verification. As a separate service it can be scaled on its own without copying the whole system.
2. **Fault isolation.** Each service maps to one business area (request and dispatch, tracking, delivery verification, notifications). If Notification is down, senders can still request and cancel couriers, and the message queue holds the events until it recovers. In a single-server design one failure would stop everything.

Security Advantage

The API Gateway is the only public entry point. It checks the identity of the sender or rider before any request is passed on, so the internal services are never exposed directly. The OTP check is kept in its own Verification Service, so a problem in another service (for example Notification) does not give access to OTP data. Each service can also be given only the permissions it needs.

Performance Benefit

Notifications are sent through a message queue instead of a direct call, so Dispatch and Verification do not wait for an SMS or push message to finish before replying to the user. Combined with scaling Tracking separately, rider matching and live location stay fast during busy hours.

Trade-off

Microservices add operational work (more services to deploy and monitor) and network calls between services. We accept this because the scenario has clearly separate parts and needs live tracking to stay responsive.